Syracuse University
School of Information Studies

Week 3
Overview of Statistics & Functions

Dr. Preeti Jagadev

Samples & Uncertainty

What is a Population:

- In data science, a population is the entire group of things or people you're interested in.

What is a Sample?

- A sample is a smaller group taken from the larger population to study or analyze.
- You can't always collect data from the whole population, as it takes too much time or money.
- So you pick a few pieces (samples) from the population.

Descriptive Statistics:

- Think of descriptive stats as simply describing what you already have.
- Imagine: "I looked at all the test scores in the class, and the average score is 82."
- That's a fact based on the data you collected. You're not guessing, predicting, or assuming anything, you're just summarizing what's in front of you.
- It's usually a summary (mean, median, range, etc.).

Inferential Statistics:

- Now let's say you only asked 10 students out of 1,000 in your school what their shoe size is and then tried to guess the average shoe size of the entire school.
- That's what inferential statistics does.
- It makes informed inferences about a big group by looking at a small part.

- When you have a set of numbers (a sample), like test scores, prices, or heights, there are two important things you want to know:

- **Measures of Central Tendency:** These tell you what's typical or average in your data.

Term	What it Means	Easy Example
Mean	The **average**	Add all the numbers, divide by how many there are. e.g., (10+20+30)/3 = 20
Median	The **middle number** when data is ordered	In [5, 10, 15], the median is 10
Mode	The number that appears **most often**	In [2, 2, 3, 5], the mode is 2

- **Measures of Dispersion (Spread of the data):** These tell you how much the values differ from each other.

Term	What it Means	Easy Example
Range	Difference between **highest and lowest** values	If scores are 60 to 90, range = 90 - 60 = 30
Variance	The **average of how far each number is from the mean** (squared)	More variation = higher variance
Standard Deviation	The **square root of the variance,** tells you how "spread out" the numbers are in a unit similar to the original data	Easier to interpret than variance

We want to understand how spread out the data is. There are different ways to measure this:

- **Range:**

Range= Maximum value−Minimum value

Example: For the group B (1, 3, 8): Range=8−1=7

- **Deviations from the Mean:**

This tells us how far each value is from the average:

Deviation = Each value−Mean

Example: Group B: 1, 3, 8; **mean** = 4, **So, the deviations are**: -3, -1, +4

- **Sum of Squares:** We square each deviation and add them up.

Example: $(-3)^2 = 9$, $(-1)^2 = 1$, $(4)^2 = 16$

Sum of squares=9+1+16=26

- **Variance**: This is the average of the squared deviations.

$$\text{Variance} = \frac{\text{Sum of squares}}{\text{Number of values}}$$

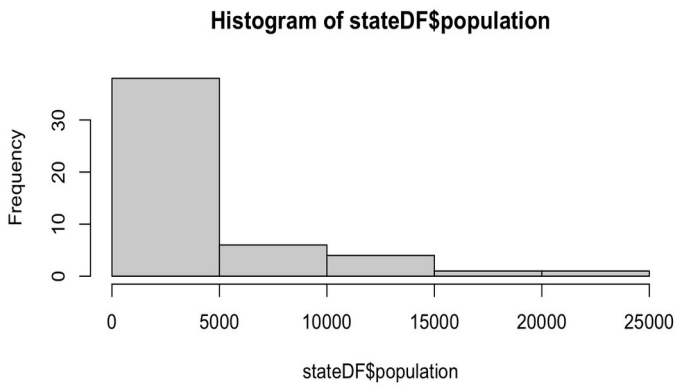
- **Standard Deviation:** This is the square root of the variance.

Histogram

- **A histogram is a way to graphically represent the distribution of your data.**
- It groups data into ranges (called bins).
- Each bar shows how many values fall into that range. The taller the bar, the more values fall in that range.
- **Example in the Slide**: The data used is **stateDF$population**, which refers to the population of each U.S. state.
- The R code to make the histogram is: ***hist(stateDF$population).***

What does the chart show?

- Most states (around 30) have a population below 5,000 (as shown by the first and tallest bar).
- Fewer states fall into higher population ranges (e.g., 5,000–10,000, 10,000–15,000, etc.).
- This means that most U.S. states have relatively low populations, while only a few states are highly populated.
- This visualization helps you quickly see patterns in population data.

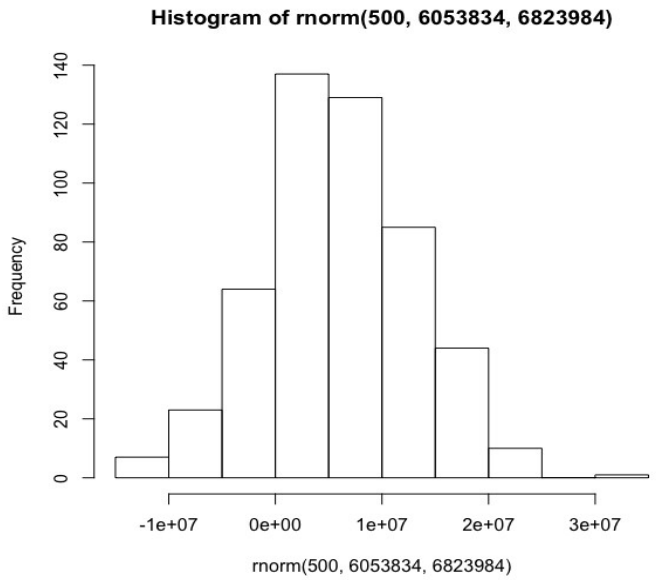


Example Histogram: More “Breaks”

- Let's create a histogram using the population data from all U.S. states with 20 breaks.
- The R command is: ***hist(stateDF$population, breaks = 20).***
- “Breaks” means how many pieces or intervals you split the data into.
- More breaks = more bars on the histogram.
- This helps you spot smaller patterns in the data that were hidden in the simpler version.

Random Distribution

- A **Random Distribution** is what you get when you collect data generated randomly.
- In this example, the histogram is created using this command: ***hist(rnorm(500, 6053834, 6823984))***
- This means ***rnorm()*** generates 500 random numbers.
- The numbers follow a ***normal distribution***.
- The ***mean*** is 6,053,834, and the ***standard deviation*** is 6,823,984.
- A normal distribution looks like a bell-shaped curve.
- Most values are close to the average (center of the bell).
- Fewer values appear as you move further from the center (the sides of the bell).



Using R Functions

- A function in R is like a tool that does a specific job for you.
- You give it something to work on (called an input or argument), and it gives you a result (called the output or return value).

Example Function: **mean()**

- This is one of the simplest and most useful functions in R.
- It calculates the average of the numbers in a vector.
- Once you create a function (or use one that already exists), you don't have to write all the steps again. This saves time and effort.
- Functions condense many steps into one simple line. Instead of writing 10 lines of code every time, you just call the function, and it does everything for you.
- Instead of long, messy code, using functions makes your code clean and organized.

Coding

```r
install.packages("tidyverse")
library(tidyverse)
```

```r
#Sorting the dataframe
arrange(USArrests, Murder)
```

```
##                 Murder Assault UrbanPop Rape
## North Dakota       0.8      45       44  7.3
## Maine              2.1      83       51  7.8
## New Hampshire      2.1      57       56  9.5
## Iowa               2.2      56       57 11.3
## Vermont            2.2      48       32 11.2
## Idaho              2.6     120       54 14.2
## Wisconsin          2.6      53       66 10.8
## Minnesota          2.7      72       66 14.9
## Utah               3.2     120       80 22.9
## Connecticut        3.3     110       77 11.1
## Rhode Island       3.4     174       87  8.3
## South Dakota       3.8      86       45 12.8
## Washington         4.0     145       73 26.2
## Nebraska           4.3     102       62 16.5
## Massachusetts      4.4     149       85 16.3
## Oregon             4.9     159       67 29.3
## Hawaii             5.3      46       83 20.2
## West Virginia      5.7      81       39  9.3
## Delaware           5.9     238       72 15.8
## Kansas             6.0     115       66 18.0
```

arrange()

- This is a function
- It means “put things in order” or “sort”

USArrests

- This is a dataset about crime rates in US states

Each row = one state

Each column = one type of crime

Murder

This is one column in the USArrests table

It shows the murder rate for each state

How it works together

- arrange() looks at the Murder column
- It reorders all states
- States with smaller murder numbers come first
- States with bigger murder numbers come later

Using Pipes:

```r
USArrests %>% arrange(Murder)
```

```r
topMurders <- USArrests %>%
      add_column(name=rownames(USArrests) ) %>%
      arrange(desc(Murder))

topMurders[1:3,]
```

add_column()
- A function that adds a new column to the table
- Here the column is called name

rownames(USArrests)
- Gets the row names (state names) of the dataset, and puts state names into that column

arrange(**desc**(Murder))
- Sorts states from highest murder rate to lowest

topMurders[1:3,]
Used to select rows 1 to 3
- These are the top 3 states and all columns

Descriptive Stats Example in R

```
stateDF <- data.frame(state=state.name, population=state.x77[,1])
stateDF$state[1]
```

```
stateDF$population[1]

## [1] 3615
mean(stateDF$population)

## [1] 4246.42
median(stateDF$population)

## [1] 2838.5
mode(stateDF$population)

## [1] "numeric"
```

state.name

- A built-in R list
- Contains the names of all 50 US states

state.x77

- A built-in R dataset
- Contains different facts about US states
- The first column of state.x77 is population.

What this whole line does

Make a table with state names and populations, then show the first state.”

gsub (Global substitution) function

- It replaces all occurrences of a pattern (like a word or character) in a string with something else.
- Works on text (strings)
- Suppose you have a small data frame of people with emails:

```
people <- data.frame(
        name = c("Alice", "Bob"),
        email = c("alice@gmail.com", "bob@gmail.com")
)
```

Now, you want to remove @gmail.com from the email addresses.

Use gsub to remove the suffix

```
people$email <- gsub("@gmail.com", "", people$email)
```

Part	Meaning
gsub	Function that replaces text in strings
"@gmail.com"	The text you want to remove
""	Replace it with nothing (delete it)
people$email	The column of emails where we want to remove the text

A Simple Function Example:

```
myFunc <- function(x) {
  return(3 * x)
}
myFunc(4)
```

```
## [1] 12
```

- Increases the reusability of code
- Makes R code easier to read
- Reduces code repetition

Create a function to sort a data frame's columns alphabetically by column name

```r
colNameSort <- function(inputDF) {
  nameList <- colnames(inputDF) # Get the list of column names from the input data frame
  inputDF[ , order(nameList)] # Reorder the columns using alphabetical order of the names
}
testDF <- colNameSort(mtcars) # Run it on mtcars
str(testDF) # Let's see the result
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
```

- **colNameSort:** The name of the function
- **function(inputDF):** Defines a function that takes input, called inputDF
- **nameList:** Variable that now holds all column names as a character vector
- **order(nameList):** Gives the index positions of the column names in alphabetical order**. Example:** if columns are c("gear", "mpg", "cyl"), order(nameList) gives positions 3,2,1

Syracuse University
School of Information Studies

The End